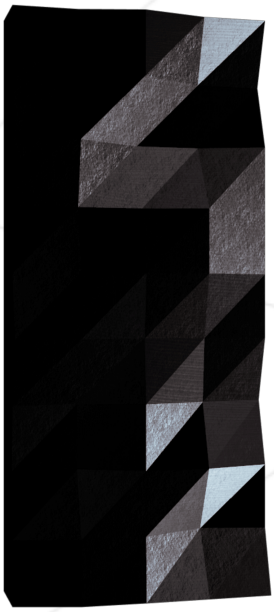

Embedded pool

Module07 : EEPROM

contact@42chips.fr

Summary: Because writing too many times at the same spot is what made cars crash

Version: 1

Chapter I

Introduction

An EEPROM (Electrically Erasable Programmable Read-Only Memory) is a type of non-volatile memory that can be used to store data on a microcontroller.

It can be written to and erased multiple times, and retains its data when the power is turned off.

EEPROMs are useful for storing data that needs to be retained even when the microcontroller is not powered, such as configuration settings or calibration data.

They are slower and have less capacity than other types of memory, such as SRAM or flash memory, but are still useful in a variety of applications.

Chapter II

General instructions

Unless explicitly stated otherwise, the following instructions will be valid for all assignments.

- The language used for this project is C.
- It is not necessary to code according to the 42 norm.
- The exercises are ordered very precisely from the simplest to the most complex. Under no circumstances will we consider or evaluate a complex exercise if a simpler one is not perfectly successful.
- You must not leave any files other than those explicitly specified by the exercise instructions in your directory during peer evaluation.
- All technical answers to your questions can be found in the `datasheets` or on the Internet. It is up to you to use and abuse these resources to understand how to complete your exercise.
- You must use the datasheet of the microcontroller provided to you and comment on the important parts of your program by indicating where you found the clues in the document, and if necessary, explaining your approach. Don't write long blocks of text, keep it clear.
- Do you have a question? Ask your neighbor to the right or left. You can ask in the dedicated channel on the Piscine's Discord, or as a last resort, ask a staff member.

Chapter III

Tutorial

	Exercise00
What a mess!	
Directory: *ex*00/	
Files to Submit: `Makefile, *.c, *.h`	
Authorized: `avr/io.h, avr/eeprom.h`	

Write a program that displays the status of the entire EEPROM on the UART in hexdump format.

```
00000000 57 68 61 74 20 64 6f 20 79 6f 75 20 63 61 6c 6c |What do you call|
00000010 20 61 20 6d 65 64 69 74 61 74 69 6e 67 20 65 6c | a meditating el|
00000020 65 63 74 72 69 63 69 61 6e 3f 0a 0a 00 00 00 00 |ectrician?......|
...
```

Do not assume the EEPROM is set at 0 when you didn't write to it yet.

	Exercise01
	Sniper !
Directory: *ex*01/	
Files to Submit: `Makefile, *.c, *.h`	
Authorized: `avr/io.h, avr/eeprom.h`	

The number of write cycles on an EEPROM is limited, but not the number of reads..

Write a command-line program on the UART.

When given a valid address on the EEPROM in hexadecimal it writes an hexadecimal value in a byte at the address to the EEPROM, then displays the EEPROM contents with the byte in red.

If the byte already had this value, nothing should happen.

```
> 10 7f
00000000 4a6f 686e 2046 6974 7a67 6572 616c 6420
00000010 7f65 6e6e 6564 792c 2064 6974 204a 6163
00000020 6b20 4b65 6e6e 6564 792c 0a63 6f6d 6d75
00000030 6ec3 a96d 656e 7420 6170 7065 6cc3 a920
00000040 4a6f 686e 2046 2e20 4b65 6e6e 6564 7920
00000050 6574 2070 6172 2073 6573 2069 6e69 7469
00000060 616c 6573 204a 464b 2c0a 6ec3 a920 6c65
00000070 2032 3920 6d61 6920 3139 3137 20c3 a020
00000080 4272 6f6f 6b6c 696e 6520 284d 6173 7361
00000090 6368 7573 6574 7473 290a 6574 206d 6f72
000000a0 7420 6173 7361 7373 696e c3a9 206c 6520
000000b0 3232 206e 6f76 656d 6272 6520 3139 3633
000000c0 20c3 a020 4461 6c6c 6173 2028 5465 7861
000000d0 7329 0a00 0000 0000 0000 0000 0000 0000
...
```

Chapter IV

From the moment I understood the weakness of my EEPROM

	Exercise02
CRC You Later	
Directory: *ex*02/	
Files to Submit: `Makefile, *.c, *.h`	
Authorized: `avr/io.h, avr/eeprom.h`	

You are developing a "Network Node" device. Each node must store its unique operational parameters in its non-volatile EEPROM. Because these nodes are deployed in the field, they must be able to detect data corruption and handle the physical wear of the memory. You must define a `struct` representing the device configuration:

- A **Magic Number**: To identify your data block.
- A **Node ID**: A 32-bit unsigned integer.
- A **Priority**: A 16-bit signed integer.
- A **Tag**: A string of 32 characters maximum (not including the null-terminator). It must only contain alphanumeric characters, dashes (-), or underscores (_).
- An **Integrity Check**: A field of at least 16 bits. The **Integrity Check** must be calculated dynamically based on the Node ID, Priority, and Tag. It must be robust enough to detect a single-bit change in any of the fields. You are free to choose your algorithm; common industrial choices include a **16-bit Additive Checksum**, **Fletcher's Checksum**, or **CRC-16** (available in `util/crc16.h`). A simple 8-bit XOR sum or a fixed value is not sufficient for this exercise.

`Hardware is unreliable. Between power brownouts and cosmic rays, your device's configuration will eventually face bit-flips. In production, engineers also include a "version" field in their structs to manage migrations.`

Wear Leveling and Reliability

EEPROM has a limited number of write cycles. To maximize the longevity of your node, you must implement the following safety measures:

- **Write Economy**: Do not overwrite a byte if the value already stored in memory is identical to the new value.
- **Verification**: Every write operation must be verified by reading the byte back immediately to ensure it was stored correctly.
- **Relocation**: If a write verification fails (the read value does not match the intended value), you must relocate the entire configuration structure to a new address.
- Your program must support at least **4 predefined slots** in the EEPROM. On startup, it must scan these locations to find the one containing the valid configuration.

Implement the following UART interface:

```
> STATUS
Node unconfigured
> SET_ID 1024
> STATUS
Node ID: 1024
Priority: 0
Slot: 4
Tag: "Unconfigured"
# (If a write failure occurs during a command)
> SET_TAG "Relay_01"
Corruption detected.
Relocating config to slot 1... Fail
Relocating config to slot 2... Success
Done.
```

- `STATUS` :
  - If no valid configuration is found in any slot: `Node unconfigured`.
  - If a magic number is found but the integrity check fails: `CRITICAL: Data corruption detected!`.
  - If valid: Display the ID, Priority, and Tag.
- `SET_ID`, `SET_PRIO`, `SET_TAG` : Updates the specific field, recalculates the integrity, and saves to EEPROM. If a write fails and all slots are exhausted after a full slot loop, print `CRITICAL EEPROM FAILURE`.
- `FACTORY_RESET` : Invalidates the current configuration.

Preparation for Evaluation

Since physical EEPROM failure is difficult to trigger on demand, you must be prepared to demonstrate your relocation logic to the evaluator.

Plan a "hook" in your code (such as a conditional branch or a specific debug macro) that allows you to simulate a verification failure. During the defense, you will be asked to trigger this failure to prove that your program correctly detects the error, relocates the configuration to the next available slot, and successfully retrieves the data from the new location after a reboot.